Profesor:

Yventz Entzana Garduño

Tarea

Practicas Segundo Parcial

Nombre:

Campos Salgado Diego Campos

Secuencia:

2CV22

Materia:

Abstraccion y Uso de datos

Fecha:

02/11/2026

Índice de prácticas

Práctica 16. Ordenamiento burbuja

Práctica 17. Ordenamiento Merge Sort

Práctica 18. Ordenamiento Quick Sort

Práctica 19. Ordenamiento indirecto burbuja

Práctica 20. Ordenamiento indirecto Merge Sort

Práctica 21. Ordenamiento indirecto Quick Sort

Práctica 1. Punteros con clases y estructuras

Práctica 2. ADT (tipo abstracto de datos)

Práctica 3. Pila estática con arreglo

Práctica 4. Cola estática con arreglo

Práctica 5. Lista estática con arreglo

Práctica 6. Pila dinámica con punteros

Práctica 7. Cola dinámica con punteros

Práctica 7 bis. Lista dinámica con punteros

Práctica 8. Pila con biblioteca del IDE

Práctica 9. Cola con biblioteca del IDE

Práctica 10. Lista con biblioteca del IDE

Práctica 16. Ordenamiento burbuja

Introducción

Esta práctica aborda ordenamiento burbuja en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar ordenamiento burbuja mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

Ordena caracteres, enteros y registros Alumno mediante el método de burbuja.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Archivo fuente original proporcionado: OrdenamientoBurbuja.cpp. Se conserva el código del estudiante sin sustituirlo por una versión generada.

Código fuente

```
#include <iostream>
#include <string>
using namespace std;

// Nuevo tipo de dato
struct Alumno {
    string nombre;
    int edad;
    float calificacion;
};

int main() {
    int opcion;
```

```
do {
    cout << "\n===== ORDENAMIENTO BURBUJA =====" << endl;
    cout << "1. Ordenar numeros enteros" << endl;
    cout << "2. Ordenar caracteres" << endl;
    cout << "3. Ordenar alumnos" << endl;
    cout << "4. Salir" << endl;
    cout << "Selecciona una opcion: ";
    cin >> opcion;

    switch (opcion) {

    case 1: {
        int numeros[5];
        int aux;

        cout << "\nIngresa 5 numeros enteros:\n";
        for (int i = 0; i < 5; i++) {
            cin >> numeros[i];
        }

        // Ordenamiento burbuja
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 4 - i; j++) {
                if (numeros[j] > numeros[j + 1]) {
                    aux = numeros[j];
                    numeros[j] = numeros[j + 1];
                    numeros[j + 1] = aux;
                }
            }
        }

        cout << "\nNumeros ordenados: ";
```

```
    for (int i = 0; i < 5; i++) {
        cout << numeros[i] << " ";
    }
    cout << endl;
    break;
}

case 2: {
    char letras[5];
    char aux;

    cout << "\nIngresa 5 caracteres:\n";
    for (int i = 0; i < 5; i++) {
        cin >> letras[i];
    }

    // Ordenamiento burbuja
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4 - i; j++) {
            if (letras[j] > letras[j + 1]) {
                aux = letras[j];
                letras[j] = letras[j + 1];
                letras[j + 1] = aux;
            }
        }
    }

    cout << "\nCaracteres ordenados: ";
    for (int i = 0; i < 5; i++) {
        cout << letras[i] << " ";
    }
    cout << endl;
```

```
    break;
}

case 3: {
    Alumno alumnos[3];
    Alumno aux;

    cout << "\nIngresa los datos de 3 alumnos:\n";

    for (int i = 0; i < 3; i++) {
        cout << "\nAlumno " << i + 1 << endl;
        cout << "Nombre: ";
        cin >> alumnos[i].nombre;
        cout << "Edad: ";
        cin >> alumnos[i].edad;
        cout << "Calificacion: ";
        cin >> alumnos[i].calificacion;
    }

    // Ordenamiento burbuja
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2 - i; j++) {
            if (alumnos[j].calificacion >
                alumnos[j + 1].calificacion) {

                aux = alumnos[j];
                alumnos[j] = alumnos[j + 1];
                alumnos[j + 1] = aux;
            }
        }
    }
```

```
            cout << "\nAlumnos ordenados por calificacion:\n";

            for (int i = 0; i < 3; i++) {
                cout << alumnos[i].nombre << " - "
                     << alumnos[i].edad << " anios - "
                     << alumnos[i].calificacion << endl;
            }
            break;
        }

        case 4:
            cout << "\nSaliendo del programa..." << endl;
            break;

        default:
            cout << "\nOpcion no valida." << endl;
        }

    } while (opcion != 4);

    return 0;
}
```

Pruebas de funcionamiento

Compilar y ejecutar el archivo en el IDE. Probar las opciones disponibles, verificar los resultados y registrar cualquier mensaje de error. Adjuntar una captura auténtica de la ejecución cuando sea requerida.

Análisis de resultados

Registrar aquí los datos de entrada, la salida obtenida y si el comportamiento observado corresponde al funcionamiento esperado de la práctica.

Práctica 17. Ordenamiento Merge Sort

Introducción

Esta práctica aborda ordenamiento merge sort en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar ordenamiento merge sort mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

Divide los datos y combina subsecuencias ordenadas.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Archivo fuente original proporcionado: OrdenamientoMergeSort.cpp. Se conserva el código del estudiante sin sustituirlo por una versión generada.

Código fuente

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// Nuevo tipo de dato
struct Alumno {
    string nombre;
    int edad;
    float calificacion;
};

// Funcion para comparar alumnos
```

```
bool comparar(Alumno a, Alumno b) {
    return a.calificacion <= b.calificacion;
}

// Funcion para comparar enteros
bool comparar(int a, int b) {
    return a <= b;
}

// Funcion para comparar caracteres
bool comparar(char a, char b) {
    return a <= b;
}

// Funcion que mezcla los elementos ordenados
template <typename T>
void mezclar(T datos[], int inicio, int medio, int fin) {

    int i = inicio;
    int j = medio + 1;
    vector<T> auxiliar;

    while (i <= medio && j <= fin) {
        if (comparar(datos[i], datos[j])) {
            auxiliar.push_back(datos[i]);
            i++;
        } else {
            auxiliar.push_back(datos[j]);
            j++;
        }
    }
```

```
    while (i <= medio) {
        auxiliar.push_back(datos[i]);
        i++;
    }

    while (j <= fin) {
        auxiliar.push_back(datos[j]);
        j++;
    }

    for (int k = 0; k < (int)auxiliar.size(); k++) {
        datos[inicio + k] = auxiliar[k];
    }
}

// Funcion principal de Merge Sort
template <typename T>
void mergeSort(T datos[], int inicio, int fin) {

    if (inicio < fin) {
        int medio = inicio + (fin - inicio) / 2;

        mergeSort(datos, inicio, medio);
        mergeSort(datos, medio + 1, fin);

        mezclar(datos, inicio, medio, fin);
    }
}

int main() {
    int opcion;
```

```
do {
    cout << "\n===== ORDENAMIENTO MERGE SORT =====\n";
    cout << "1. Ordenar numeros enteros\n";
    cout << "2. Ordenar caracteres\n";
    cout << "3. Ordenar alumnos\n";
    cout << "4. Salir\n";
    cout << "Selecciona una opcion: ";
    cin >> opcion;

    switch (opcion) {

    case 1: {
        int numeros[5];

        cout << "\nIngresa 5 numeros enteros:\n";

        for (int i = 0; i < 5; i++) {
            cin >> numeros[i];
        }

        mergeSort(numeros, 0, 4);

        cout << "\nNumeros ordenados: ";
        for (int i = 0; i < 5; i++) {
            cout << numeros[i] << " ";
        }
        cout << endl;
        break;
    }

    case 2: {
        char letras[5];
```

```
    cout << "\nIngresa 5 caracteres:\n";

    for (int i = 0; i < 5; i++) {
        cin >> letras[i];
    }

    mergeSort(letras, 0, 4);

    cout << "\nCaracteres ordenados: ";
    for (int i = 0; i < 5; i++) {
        cout << letras[i] << " ";
    }
    cout << endl;
    break;
}

case 3: {
    Alumno alumnos[3];

    cout << "\nIngresa los datos de 3 alumnos:\n";

    for (int i = 0; i < 3; i++) {
        cout << "\nAlumno " << i + 1 << endl;

        cout << "Nombre: ";
        cin >> alumnos[i].nombre;

        cout << "Edad: ";
        cin >> alumnos[i].edad;

        cout << "Calificacion: ";
```

```
            cin >> alumnos[i].calificacion;
        }

        mergeSort(alumnos, 0, 2);

        cout << "\nAlumnos ordenados por calificacion:\n";

        for (int i = 0; i < 3; i++) {
            cout << alumnos[i].nombre << " - "
                 << alumnos[i].edad << " anios - "
                 << alumnos[i].calificacion << endl;
        }
        break;
    }

    case 4:
        cout << "\nSaliendo del programa...\n";
        break;

    default:
        cout << "\nOpcion no valida.\n";
    }

  } while (opcion != 4);

  return 0;
}
```

Pruebas de funcionamiento

Compilar y ejecutar el archivo en el IDE. Probar las opciones disponibles, verificar los resultados y registrar cualquier mensaje de error. Adjuntar una captura auténtica de la ejecución cuando sea requerida.

Análisis de resultados

Registrar aquí los datos de entrada, la salida obtenida y si el comportamiento observado corresponde al funcionamiento esperado de la práctica.

Práctica 18. Ordenamiento Quick Sort

Introducción

Esta práctica aborda ordenamiento quick sort en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar ordenamiento quick sort mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

Ordena mediante particiones alrededor de un pivote.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Archivo fuente original proporcionado: OrdenaminetoQuickSort.cpp. Se conserva el código del estudiante sin sustituirlo por una versión generada.

Código fuente

```
#include <iostream>
#include <string>
using namespace std;

// Nuevo tipo de dato
struct Alumno {
    string nombre;
    int edad;
    float calificacion;
};

// Comparaciones para cada tipo de dato
```

```
bool menor(int a, int b) {
    return a < b;
}

bool menor(char a, char b) {
    return a < b;
}

bool menor(Alumno a, Alumno b) {
    return a.calificacion < b.calificacion;
}

// Intercambiar dos elementos
template <typename T>
void intercambiar(T &a, T &b) {
    T aux = a;
    a = b;
    b = aux;
}

// Funcion para dividir usando un pivote
template <typename T>
int particion(T datos[], int inicio, int fin) {

    T pivote = datos[fin];
    int i = inicio - 1;

    cout << "\nParticionando del indice "
         << inicio << " al " << fin << endl;
    cout << "Pivote seleccionado: ";

    if constexpr (false) {
```

```
        // No se utiliza
    }

    for (int j = inicio; j < fin; j++) {
        if (!menor(pivote, datos[j])) {
            i++;
            intercambiar(datos[i], datos[j]);
        }
    }

    intercambiar(datos[i + 1], datos[fin]);

    cout << "Pivote colocado en posicion "
         << i + 1 << endl;

    return i + 1;
}

// Funcion principal Quick Sort
template <typename T>
void quickSort(T datos[], int inicio, int fin) {

    if (inicio < fin) {
        int posicion = particion(datos, inicio, fin);

        quickSort(datos, inicio, posicion - 1);
        quickSort(datos, posicion + 1, fin);
    }
}

int main() {
    int opcion;
```

```
do {
    cout << "\n===== ORDENAMIENTO QUICK SORT =====\n";
    cout << "1. Ordenar numeros enteros\n";
    cout << "2. Ordenar caracteres\n";
    cout << "3. Ordenar alumnos\n";
    cout << "4. Salir\n";
    cout << "Selecciona una opcion: ";
    cin >> opcion;

    switch (opcion) {

    case 1: {
        int numeros[5];

        cout << "\nIngresa 5 numeros enteros:\n";

        for (int i = 0; i < 5; i++) {
            cin >> numeros[i];
        }

        quickSort(numeros, 0, 4);

        cout << "\nNumeros ordenados: ";
        for (int i = 0; i < 5; i++) {
            cout << numeros[i] << " ";
        }
        cout << endl;
        break;
    }

    case 2: {
```

```
    char letras[5];

    cout << "\nIngresa 5 caracteres:\n";

    for (int i = 0; i < 5; i++) {
        cin >> letras[i];
    }

    quickSort(letras, 0, 4);

    cout << "\nCaracteres ordenados: ";
    for (int i = 0; i < 5; i++) {
        cout << letras[i] << " ";
    }
    cout << endl;
    break;
}

case 3: {
    Alumno alumnos[3];

    cout << "\nIngresa los datos de 3 alumnos:\n";

    for (int i = 0; i < 3; i++) {
        cout << "\nAlumno " << i + 1 << endl;

        cout << "Nombre: ";
        cin >> alumnos[i].nombre;

        cout << "Edad: ";
        cin >> alumnos[i].edad;
```

```
            cout << "Calificacion: ";
            cin >> alumnos[i].calificacion;
        }

        quickSort(alumnos, 0, 2);

        cout << "\nAlumnos ordenados por calificacion:\n";

        for (int i = 0; i < 3; i++) {
            cout << alumnos[i].nombre << " - "
                 << alumnos[i].edad << " anios - "
                 << alumnos[i].calificacion << endl;
        }
        break;
    }

    case 4:
        cout << "\nSaliendo del programa...\n";
        break;

    default:
        cout << "\nOpcion no valida.\n";
    }

} while (opcion != 4);

return 0;
}
```

Pruebas de funcionamiento

Compilar y ejecutar el archivo en el IDE. Probar las opciones disponibles, verificar los resultados y registrar cualquier mensaje de error. Adjuntar una captura auténtica de la ejecución cuando sea requerida.

Análisis de resultados

Registrar aquí los datos de entrada, la salida obtenida y si el comportamiento observado corresponde al funcionamiento esperado de la práctica.

Práctica 19. Ordenamiento indirecto burbuja

Introducción

Esta práctica aborda ordenamiento indirecto burbuja en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar ordenamiento indirecto burbuja mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

Emplea referencias o punteros para manejar el ordenamiento indirecto.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Archivo fuente original proporcionado: OrdenaminetoIndirecta.cpp. Se conserva el código del estudiante sin sustituirlo por una versión generada.

Código fuente

```
#include <iostream>
#include <string>
using namespace std;

// Nuevo tipo de dato
struct Alumno {
    string nombre;
    int edad;
    float calificacion;
};

// Comparaciones para cada tipo de dato
bool mayor(int a, int b) {
```

```
    return a > b;
}

bool mayor(char a, char b) {
    return a > b;
}

bool mayor(const Alumno &a, const Alumno &b) {
    return a.calificacion > b.calificacion;
}

// Ordenamiento Burbuja indirecto
template <typename T>
void burbujaIndirecta(T datos[], int indices[], int n) {

    // Inicializar los indices
    for (int i = 0; i < n; i++) {
        indices[i] = i;
    }

    // Ordenar los indices con Burbuja
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {

            if (mayor(datos[indices[j]],
                    datos[indices[j + 1]])) {

                int aux = indices[j];
                indices[j] = indices[j + 1];
                indices[j + 1] = aux;
            }
        }
```

```
    }
}

// Mostrar indices ordenados
void mostrarIndices(int indices[], int n) {
    cout << "\nIndices ordenados: ";

    for (int i = 0; i < n; i++) {
        cout << indices[i] << " ";
    }
    cout << endl;
}

int main() {
    int opcion;

    do {
        cout << "\n===== BURBUJA INDIRECTA =====\n";
        cout << "1. Ordenar numeros enteros\n";
        cout << "2. Ordenar caracteres\n";
        cout << "3. Ordenar alumnos\n";
        cout << "4. Salir\n";
        cout << "Selecciona una opcion: ";
        cin >> opcion;

        switch (opcion) {

        case 1: {
            int numeros[5];
            int indices[5];

            cout << "\nIngresa 5 numeros enteros:\n";
```

```
    for (int i = 0; i < 5; i++) {
        cin >> numeros[i];
    }

    burbujaIndirecta(numeros, indices, 5);

    cout << "\nNumeros originales: ";
    for (int i = 0; i < 5; i++) {
        cout << numeros[i] << " ";
    }

    mostrarIndices(indices, 5);

    cout << "Numeros ordenados: ";
    for (int i = 0; i < 5; i++) {
        cout << numeros[indices[i]] << " ";
    }
    cout << endl;
    break;
}

case 2: {
    char letras[5];
    int indices[5];

    cout << "\nIngresa 5 caracteres:\n";

    for (int i = 0; i < 5; i++) {
        cin >> letras[i];
    }
```

```
    burbujaIndirecta(letras, indices, 5);

    cout << "\nCaracteres originales: ";
    for (int i = 0; i < 5; i++) {
        cout << letras[i] << " ";
    }

    mostrarIndices(indices, 5);

    cout << "Caracteres ordenados: ";
    for (int i = 0; i < 5; i++) {
        cout << letras[indices[i]] << " ";
    }
    cout << endl;
    break;
}

case 3: {
    Alumno alumnos[3];
    int indices[3];

    cout << "\nIngresa los datos de 3 alumnos:\n";

    for (int i = 0; i < 3; i++) {
        cout << "\nAlumno " << i + 1 << endl;

        cout << "Nombre: ";
        cin >> alumnos[i].nombre;

        cout << "Edad: ";
        cin >> alumnos[i].edad;
```

```
            cout << "Calificacion: ";
            cin >> alumnos[i].calificacion;
        }

        burbujaIndirecta(alumnos, indices, 3);

        cout << "\nAlumnos originales:\n";
        for (int i = 0; i < 3; i++) {
            cout << alumnos[i].nombre << " - "
                 << alumnos[i].edad << " anios - "
                 << alumnos[i].calificacion << endl;
        }

        mostrarIndices(indices, 3);

        cout << "\nAlumnos ordenados por calificacion:\n";

        for (int i = 0; i < 3; i++) {
            int pos = indices[i];

            cout << alumnos[pos].nombre << " - "
                 << alumnos[pos].edad << " anios - "
                 << alumnos[pos].calificacion << endl;
        }
        break;
    }

    case 4:
        cout << "\nSaliendo del programa...\n";
        break;

    default:
```

```
            cout << "\nOpcion no valida.\n";
        }

    } while (opcion != 4);

    return 0;
}
```

Pruebas de funcionamiento

Compilar y ejecutar el archivo en el IDE. Probar las opciones disponibles, verificar los resultados y registrar cualquier mensaje de error. Adjuntar una captura auténtica de la ejecución cuando sea requerida.

Análisis de resultados

Registrar aquí los datos de entrada, la salida obtenida y si el comportamiento observado corresponde al funcionamiento esperado de la práctica.

Práctica 20. Ordenamiento indirecto Merge Sort

Introducción

Esta práctica aborda ordenamiento indirecto merge sort en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar ordenamiento indirecto merge sort mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

Implementa ordenamiento indirecto mediante mezcla.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Archivo fuente original proporcionado: OrdenamientoIndirectaMergeSort.cpp. Se conserva el código del estudiante sin sustituirlo por una versión generada.

Código fuente

```
#include <iostream>
#include <string>
using namespace std;

// Nuevo tipo de dato
struct Alumno {
    string nombre;
    int edad;
    float calificacion;
};

// Comparar numeros enteros
bool menorIgual(int a, int b) {
```

```
    return a <= b;
}

// Comparar caracteres
bool menorIgual(char a, char b) {
    return a <= b;
}

// Comparar alumnos por calificacion
bool menorIgual(const Alumno &a, const Alumno &b) {
    return a.calificacion <= b.calificacion;
}

// Funcion para mezclar indices ordenados
template <typename T>
void mezclarIndirecto(
    const T datos[], int indices[],
    int inicio, int medio, int fin) {

    int n = fin - inicio + 1;
    int* auxiliar = new int[n];

    int i = inicio;
    int j = medio + 1;
    int k = 0;

    while (i <= medio && j <= fin) {

        if (menorIgual(datos[indices[i]],
                       datos[indices[j]])) {
            auxiliar[k] = indices[i];
            i++;
```

```
        } else {
            auxiliar[k] = indices[j];
            j++;
        }
        k++;
    }

    while (i <= medio) {
        auxiliar[k] = indices[i];
        i++;
        k++;
    }

    while (j <= fin) {
        auxiliar[k] = indices[j];
        j++;
        k++;
    }

    for (int x = 0; x < n; x++) {
        indices[inicio + x] = auxiliar[x];
    }

    delete[] auxiliar;
}

// Funcion Merge Sort indirecto
template <typename T>
void mergeSortIndirecto(
    const T datos[], int indices[],
    int inicio, int fin) {
```

```
    if (inicio < fin) {

        int medio = inicio + (fin - inicio) / 2;

        mergeSortIndirecto(
            datos, indices, inicio, medio);

        mergeSortIndirecto(
            datos, indices, medio + 1, fin);

        mezclarIndirecto(
            datos, indices, inicio, medio, fin);
    }
}

// Inicializar los indices
template <typename T>
void ordenarIndirecto(
    const T datos[], int indices[], int n) {

    for (int i = 0; i < n; i++) {
        indices[i] = i;
    }

    mergeSortIndirecto(datos, indices, 0, n - 1);
}

// Mostrar indices
void mostrarIndices(int indices[], int n) {

    cout << "\nIndices ordenados: ";
```

```
    for (int i = 0; i < n; i++) {
        cout << indices[i] << " ";
    }

    cout << endl;
}

int main() {
    int opcion;

    do {
        cout << "\n===== MERGE SORT INDIRECTO =====\n";
        cout << "1. Ordenar numeros enteros\n";
        cout << "2. Ordenar caracteres\n";
        cout << "3. Ordenar alumnos\n";
        cout << "4. Salir\n";
        cout << "Selecciona una opcion: ";
        cin >> opcion;

        switch (opcion) {

        case 1: {
            int numeros[5];
            int indices[5];

            cout << "\nIngresa 5 numeros enteros:\n";

            for (int i = 0; i < 5; i++) {
                cin >> numeros[i];
            }

            ordenarIndirecto(numeros, indices, 5);
```

```
    cout << "\nNumeros originales: ";
    for (int i = 0; i < 5; i++) {
        cout << numeros[i] << " ";
    }

    mostrarIndices(indices, 5);

    cout << "Numeros ordenados: ";
    for (int i = 0; i < 5; i++) {
        cout << numeros[indices[i]] << " ";
    }
    cout << endl;
    break;
}

case 2: {
    char letras[5];
    int indices[5];

    cout << "\nIngresa 5 caracteres:\n";

    for (int i = 0; i < 5; i++) {
        cin >> letras[i];
    }

    ordenarIndirecto(letras, indices, 5);

    cout << "\nCaracteres originales: ";
    for (int i = 0; i < 5; i++) {
        cout << letras[i] << " ";
    }
```

```
    mostrarIndices(indices, 5);

    cout << "Caracteres ordenados: ";
    for (int i = 0; i < 5; i++) {
        cout << letras[indices[i]] << " ";
    }
    cout << endl;
    break;
}

case 3: {
    Alumno alumnos[3];
    int indices[3];

    cout << "\nIngresa los datos de 3 alumnos:\n";

    for (int i = 0; i < 3; i++) {
        cout << "\nAlumno " << i + 1 << endl;

        cout << "Nombre: ";
        cin >> alumnos[i].nombre;

        cout << "Edad: ";
        cin >> alumnos[i].edad;

        cout << "Calificacion: ";
        cin >> alumnos[i].calificacion;
    }

    ordenarIndirecto(alumnos, indices, 3);
```

```
            cout << "\nAlumnos originales:\n";

            for (int i = 0; i < 3; i++) {
                cout << alumnos[i].nombre << " - "
                     << alumnos[i].edad << " anios - "
                     << alumnos[i].calificacion << endl;
            }

            mostrarIndices(indices, 3);

            cout << "\nAlumnos ordenados por calificacion:\n";

            for (int i = 0; i < 3; i++) {
                int pos = indices[i];

                cout << alumnos[pos].nombre << " - "
                     << alumnos[pos].edad << " anios - "
                     << alumnos[pos].calificacion << endl;
            }
            break;
        }

        case 4:
            cout << "\nSaliendo del programa...\n";
            break;

        default:
            cout << "\nOpcion no valida.\n";
        }

    } while (opcion != 4);
```

```
    return 0;
}
```

Pruebas de funcionamiento

Compilar y ejecutar el archivo en el IDE. Probar las opciones disponibles, verificar los resultados y registrar cualquier mensaje de error. Adjuntar una captura auténtica de la ejecución cuando sea requerida.

Análisis de resultados

Registrar aquí los datos de entrada, la salida obtenida y si el comportamiento observado corresponde al funcionamiento esperado de la práctica.

Práctica 21. Ordenamiento indirecto Quick Sort

Introducción

Esta práctica aborda ordenamiento indirecto quick sort en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar ordenamiento indirecto quick sort mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

Implementa ordenamiento indirecto mediante particiones.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Archivo fuente original proporcionado: OrdenamientoIndirectaQuickSort.cpp. Se conserva el código del estudiante sin sustituirlo por una versión generada.

Código fuente

```
#include <iostream>
#include <string>
using namespace std;

// Nuevo tipo de dato
struct Alumno {
    string nombre;
    int edad;
    float calificacion;
};

// Comparar numeros enteros
bool menorIgual(int a, int b) {
```

```
    return a <= b;
}

// Comparar caracteres
bool menorIgual(char a, char b) {
    return a <= b;
}

// Comparar alumnos por calificacion
bool menorIgual(const Alumno &a, const Alumno &b) {
    return a.calificacion <= b.calificacion;
}

// Intercambiar indices
void intercambiar(int &a, int &b) {
    int aux = a;
    a = b;
    b = aux;
}

// Funcion para realizar la particion indirecta
template <typename T>
int particionIndirecta(
    const T datos[], int indices[],
    int inicio, int fin) {

    int pivote = indices[fin];
    int i = inicio - 1;

    for (int j = inicio; j < fin; j++) {

        if (menorIgual(datos[indices[j]],
```

```
                    datos[pivote])) {

            i++;
            intercambiar(indices[i], indices[j]);
        }
    }

    intercambiar(indices[i + 1], indices[fin]);

    cout << "Pivote colocado en posicion: "
         << i + 1 << endl;

    return i + 1;
}

// Funcion Quick Sort indirecto
template <typename T>
void quickSortIndirecto(
    const T datos[], int indices[],
    int inicio, int fin) {

    if (inicio < fin) {

        int posicion = particionIndirecta(
            datos, indices, inicio, fin);

        quickSortIndirecto(
            datos, indices, inicio, posicion - 1);

        quickSortIndirecto(
            datos, indices, posicion + 1, fin);
    }
```

```
}

// Inicializar los indices
template <typename T>
void ordenarIndirecto(
    const T datos[], int indices[], int n) {

    for (int i = 0; i < n; i++) {
        indices[i] = i;
    }

    quickSortIndirecto(datos, indices, 0, n - 1);
}

// Mostrar indices ordenados
void mostrarIndices(int indices[], int n) {

    cout << "\nIndices ordenados: ";

    for (int i = 0; i < n; i++) {
        cout << indices[i] << " ";
    }

    cout << endl;
}

int main() {
    int opcion;

    do {
        cout << "\n===== QUICK SORT INDIRECTO =====\n";
        cout << "1. Ordenar numeros enteros\n";
```

```
cout << "2. Ordenar caracteres\n";
cout << "3. Ordenar alumnos\n";
cout << "4. Salir\n";
cout << "Selecciona una opcion: ";
cin >> opcion;

switch (opcion) {

case 1: {
    int numeros[5];
    int indices[5];

    cout << "\nIngresa 5 numeros enteros:\n";

    for (int i = 0; i < 5; i++) {
        cin >> numeros[i];
    }

    cout << "\nProceso Quick Sort:\n";
    ordenarIndirecto(numeros, indices, 5);

    cout << "\nNumeros originales: ";
    for (int i = 0; i < 5; i++) {
        cout << numeros[i] << " ";
    }

    mostrarIndices(indices, 5);

    cout << "Numeros ordenados: ";
    for (int i = 0; i < 5; i++) {
        cout << numeros[indices[i]] << " ";
    }
```

```
    cout << endl;
    break;
}

case 2: {
    char letras[5];
    int indices[5];

    cout << "\nIngresa 5 caracteres:\n";

    for (int i = 0; i < 5; i++) {
        cin >> letras[i];
    }

    cout << "\nProceso Quick Sort:\n";
    ordenarIndirecto(letras, indices, 5);

    cout << "\nCaracteres originales: ";
    for (int i = 0; i < 5; i++) {
        cout << letras[i] << " ";
    }

    mostrarIndices(indices, 5);

    cout << "Caracteres ordenados: ";
    for (int i = 0; i < 5; i++) {
        cout << letras[indices[i]] << " ";
    }
    cout << endl;
    break;
}
```

```
case 3: {
    Alumno alumnos[3];
    int indices[3];

    cout << "\nIngresa los datos de 3 alumnos:\n";

    for (int i = 0; i < 3; i++) {
        cout << "\nAlumno " << i + 1 << endl;

        cout << "Nombre: ";
        cin >> alumnos[i].nombre;

        cout << "Edad: ";
        cin >> alumnos[i].edad;

        cout << "Calificacion: ";
        cin >> alumnos[i].calificacion;
    }

    cout << "\nProceso Quick Sort:\n";
    ordenarIndirecto(alumnos, indices, 3);

    cout << "\nAlumnos originales:\n";

    for (int i = 0; i < 3; i++) {
        cout << alumnos[i].nombre << " - "
             << alumnos[i].edad << " anios - "
             << alumnos[i].calificacion << endl;
    }

    mostrarIndices(indices, 3);
```

```
            cout << "\nAlumnos ordenados por calificacion:\n";

            for (int i = 0; i < 3; i++) {
                int pos = indices[i];

                cout << alumnos[pos].nombre << " - "
                    << alumnos[pos].edad << " anios - "
                    << alumnos[pos].calificacion << endl;
            }
            break;
        }

        case 4:
            cout << "\nSaliendo del programa...\n";
            break;

        default:
            cout << "\nOpcion no valida.\n";
        }

    } while (opcion != 4);

    return 0;
}
```

Pruebas de funcionamiento

Compilar y ejecutar el archivo en el IDE. Probar las opciones disponibles, verificar los resultados y registrar cualquier mensaje de error. Adjuntar una captura auténtica de la ejecución cuando sea requerida.

Análisis de resultados

Registrar aquí los datos de entrada, la salida obtenida y si el comportamiento observado corresponde al funcionamiento esperado de la práctica.

Práctica 1. Punteros con clases y estructuras

Introducción

Esta práctica aborda punteros con clases y estructuras en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar punteros con clases y estructuras mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

Emplea punteros o referencias al mismo tipo dentro de estructuras o clases.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Archivo fuente original proporcionado: PunterosManejoIndirectoConClasesyEstructuras.cpp. Se conserva el código del estudiante sin sustituirlo por una versión generada.

Código fuente

```
#include <iostream>
#include <string>
using namespace std;

// Estructura con puntero al mismo tipo
struct Alumno {
    string nombre;
    int edad;
    float calificacion;
    Alumno* siguiente;
};
```

```
int main() {

    Alumno alumnos[3];
    Alumno* inicio = nullptr;

    cout << "===== REGISTRO DE ALUMNOS =====\n";

    for (int i = 0; i < 3; i++) {
        cout << "\nAlumno " << i + 1 << endl;

        cout << "Nombre: ";
        cin >> alumnos[i].nombre;

        cout << "Edad: ";
        cin >> alumnos[i].edad;

        cout << "Calificacion: ";
        cin >> alumnos[i].calificacion;

        alumnos[i].siguiente = nullptr;
    }

    // Enlazar los alumnos
    for (int i = 0; i < 2; i++) {
        alumnos[i].siguiente = &alumnos[i + 1];
    }

    inicio = &alumnos[0];

    cout << "\n===== LISTA ORIGINAL =====\n";

    Alumno* actual = inicio;
```

```
while (actual != nullptr) {
    cout << actual->nombre << " - "
         << actual->calificacion << endl;

    actual = actual->siguiente;
}

// Ordenamiento indirecto de los enlaces
Alumno* ordenado = nullptr;

while (inicio != nullptr) {

    Alumno* nodo = inicio;
    inicio = inicio->siguiente;

    if (ordenado == nullptr ||
        nodo->calificacion <
        ordenado->calificacion) {

        nodo->siguiente = ordenado;
        ordenado = nodo;

    } else {

        Alumno* aux = ordenado;

        while (aux->siguiente != nullptr &&
               aux->siguiente->calificacion <=
               nodo->calificacion) {

            aux = aux->siguiente;
```

```
            }

            nodo->siguiente = aux->siguiente;
            aux->siguiente = nodo;
        }
    }

    cout << "\n===== LISTA ORDENADA =====\n";

    actual = ordenado;

    while (actual != nullptr) {

        cout << actual->nombre << " - "
             << actual->calificacion << endl;

        actual = actual->siguiente;
    }

    return 0;
}
```

Pruebas de funcionamiento

Compilar y ejecutar el archivo en el IDE. Probar las opciones disponibles, verificar los resultados y registrar cualquier mensaje de error. Adjuntar una captura auténtica de la ejecución cuando sea requerida.

Análisis de resultados

Registrar aquí los datos de entrada, la salida obtenida y si el comportamiento observado corresponde al funcionamiento esperado de la práctica.

Práctica 2. ADT (tipo abstracto de datos)

Introducción

Esta práctica aborda adt (tipo abstracto de datos) en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar adt (tipo abstracto de datos) mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

No se encontró un archivo .cpp identificable para esta práctica.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Código fuente pendiente: no se encontró un archivo que corresponda inequívocamente a esta práctica en el ZIP proporcionado. Se deja el apartado identificado para integrar el código original sin sustituirlo por otro.

Código fuente pendiente

Pruebas de funcionamiento

Pendientes de incorporar al recibir el código original.

Práctica 3. Pila estática con arreglo

Introducción

Esta práctica aborda pila estática con arreglo en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar pila estática con arreglo mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

No se encontró un archivo .cpp identificable como pila estática con arreglo.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Código fuente pendiente: no se encontró un archivo que corresponda inequívocamente a esta práctica en el ZIP proporcionado. Se deja el apartado identificado para integrar el código original sin sustituirlo por otro.

Código fuente pendiente

Pruebas de funcionamiento

Pendientes de incorporar al recibir el código original.

Práctica 4. Cola estática con arreglo

Introducción

Esta práctica aborda cola estática con arreglo en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar cola estática con arreglo mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

Gestiona una cola de capacidad limitada mediante un arreglo.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Archivo fuente original proporcionado: Colas.cpp. Se conserva el código del estudiante sin sustituirlo por una versión generada.

Código fuente

```
//Programa colas con decimales
#include <iostream>
#include <limits>
using namespace std;

const int LIMITE = 10;



struct Nodo {
   float dato;
   Nodo* siguiente;
};
```

```
void insertar(Nodo*& frente, Nodo*& final, float valor) {
    Nodo* nuevo = new Nodo;
    nuevo->dato = valor;
    nuevo->siguiente = nullptr;

    if (frente == nullptr) {
        frente = nuevo;
        final = nuevo;
    } else {
        final->siguiente = nuevo;
        final = nuevo;
    }

    cout << "Elemento insertado correctamente.\n";
}

void eliminar(Nodo*& frente, Nodo*& final) {
    if (frente == nullptr) {
        cout << "La cola esta vacia.\n";
        return;
    }

    Nodo* auxiliar = frente;

    cout << "Elemento eliminado: " << frente->dato << endl;

    frente = frente->siguiente;

    if (frente == nullptr) {
        final = nullptr;
    }
```

```
    delete auxiliar;
}

void mostrar(Nodo* frente) {
    if (frente == nullptr) {
        cout << "La cola esta vacia.\n";
        return;
    }

    cout << "\nElementos de la cola:\n";

    while (frente != nullptr) {
        cout << frente->dato << " ";
        frente = frente->siguiente;
    }

    cout << endl;
}

int main() {
    Nodo* frente = nullptr;
    Nodo* final = nullptr;

    int opcion;
    float valor;

    do {
        cout << "\n--- COLA DINAMICA ---\n";
        cout << "1. Insertar\n";
        cout << "2. Eliminar\n";
        cout << "3. Mostrar\n";
        cout << "4. Salir\n";
```

```
cout << "Seleccione una opcion: ";

if (!(cin >> opcion)) {
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout << "Opcion no valida.\n";
    continue;
}

switch (opcion) {
    case 1:
        cout << "Ingrese un numero entero o decimal: ";

        if (cin >> valor) {
            insertar(frente, final, valor);
        } else {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Numero no valido.\n";
        }
        break;

    case 2:
        eliminar(frente, final);
        break;

    case 3:
        mostrar(frente);
        break;

    case 4:
        cout << "Saliendo...\n";
```

```
            break;

        default:
            cout << "Opcion no valida.\n";
        }

    } while (opcion != 4);

        while (frente != nullptr) {
        Nodo* auxiliar = frente;
        frente = frente->siguiente;
        delete auxiliar;
    }

    final = nullptr;

    return 0;
}
```

Pruebas de funcionamiento

Compilar y ejecutar el archivo en el IDE. Probar las opciones disponibles, verificar los resultados y registrar cualquier mensaje de error. Adjuntar una captura auténtica de la ejecución cuando sea requerida.

Análisis de resultados

Registrar aquí los datos de entrada, la salida obtenida y si el comportamiento observado corresponde al funcionamiento esperado de la práctica.

Práctica 5. Lista estática con arreglo

Introducción

Esta práctica aborda lista estática con arreglo en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar lista estática con arreglo mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

No se encontró un archivo .cpp identificable como lista estática con arreglo.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Código fuente pendiente: no se encontró un archivo que corresponda inequívocamente a esta práctica en el ZIP proporcionado. Se deja el apartado identificado para integrar el código original sin sustituirlo por otro.

Código fuente pendiente

Pruebas de funcionamiento

Pendientes de incorporar al recibir el código original.

Práctica 6. Pila dinámica con punteros

Introducción

Esta práctica aborda pila dinámica con punteros en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar pila dinámica con punteros mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

Gestiona una pila mediante nodos enlazados y punteros.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Archivo fuente original proporcionado: PilasPunteros.cpp. Se conserva el código del estudiante sin sustituirlo por una versión generada.

Código fuente

```
//Programa pilas
#include <iostream>
using namespace std;

const int LIMITE = 10;



struct Nodo {
   int dato;
   Nodo*  siguiente;
};

void insertar(Nodo*& cima, int valor) {
```

```
    Nodo* nuevo = new Nodo;
    nuevo->dato = valor;
    nuevo->siguiente = cima;
    cima = nuevo;

    cout << "Elemento insertado correctamente.\n";
}

void eliminar(Nodo*& cima) {
    if (cima == nullptr) {
        cout << "La pila esta vacia.\n";
        return;
    }

    Nodo* auxiliar = cima;
    cout << "Elemento eliminado: " << cima->dato << endl;

    cima = cima->siguiente;
    delete auxiliar;
}

void mostrar(Nodo* cima) {
    if (cima == nullptr) {
        cout << "La pila esta vacia.\n";
        return;
    }

    cout << "Elementos de la pila:\n";

    while (cima != nullptr) {
        cout << cima->dato << endl;
        cima = cima->siguiente;
```

```
    }
}

int main() {
    Nodo* cima = nullptr;
    int opcion, valor;

    do {
        cout << "\n--- PILA DINAMICA ---\n";
        cout << "1. Insertar\n";
        cout << "2. Eliminar\n";
        cout << "3. Mostrar\n";
        cout << "4. Salir\n";
        cout << "Seleccione una opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                cout << "Ingrese un numero: ";
                cin >> valor;
                insertar(cima, valor);
                break;

            case 2:
                eliminar(cima);
                break;

            case 3:
                mostrar(cima);
                break;

            case 4:
```

```
            cout << "Saliendo...\n";
            break;

        default:
            cout << "Opcion no valida.\n";
        }

    } while (opcion != 4);

    while (cima != nullptr) {
        Nodo* auxiliar = cima;
        cima = cima->siguiente;
        delete auxiliar;
    }

    return 0;
}
```

Pruebas de funcionamiento

Compilar y ejecutar el archivo en el IDE. Probar las opciones disponibles, verificar los resultados y registrar cualquier mensaje de error. Adjuntar una captura auténtica de la ejecución cuando sea requerida.

Análisis de resultados

Registrar aquí los datos de entrada, la salida obtenida y si el comportamiento observado corresponde al funcionamiento esperado de la práctica.

Práctica 7. Cola dinámica con punteros

Introducción

Esta práctica aborda cola dinámica con punteros en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar cola dinámica con punteros mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

Gestiona una cola dinámica mediante nodos y referencias a punteros.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Archivo fuente original proporcionado: ColaDinamica.cpp. Se conserva el código del estudiante sin sustituirlo por una versión generada.

Código fuente

```
#include <iostream>
using namespace std;

struct Nodo {
    float dato;
    Nodo* siguiente;
};

void insertar(Nodo*& frente, Nodo*& final, int& cantidad) {
    if (cantidad >= 10) {
        cout << "La cola esta llena.\n";
        return;
    }
```

```
    float valor;
    cout << "Ingresa un numero: ";
    cin >> valor;

    Nodo* nuevo = new Nodo;
    nuevo->dato = valor;
    nuevo->siguiente = nullptr;

    if (frente == nullptr) {
        frente = nuevo;
        final = nuevo;
    } else {
        final->siguiente = nuevo;
        final = nuevo;
    }

    cantidad++;
    cout << "Elemento insertado.\n";
}

void eliminar(Nodo*& frente, Nodo*& final, int& cantidad) {
    if (frente == nullptr) {
        cout << "La cola esta vacia.\n";
        return;
    }

    Nodo* auxiliar = frente;
    cout << "Elemento eliminado: " << auxiliar->dato << endl;

    frente = frente->siguiente;
    delete auxiliar;
```

```
    cantidad--;

    if (frente == nullptr) {
        final = nullptr;
    }
}

void mostrar(Nodo* frente) {
    if (frente == nullptr) {
        cout << "La cola esta vacia.\n";
        return;
    }

    cout << "Elementos de la cola: ";
    while (frente != nullptr) {
        cout << frente->dato << " ";
        frente = frente->siguiente;
    }
    cout << endl;
}

int main() {
    Nodo* frente = nullptr;
    Nodo* final = nullptr;
    int cantidad = 0;
    int opcion;

    do {
        cout << "\n--- COLA DINAMICA ---\n";
        cout << "1. Insertar\n";
        cout << "2. Eliminar\n";
        cout << "3. Mostrar\n";
```

```
        cout << "4. Salir\n";
        cout << "Opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                insertar(frente, final, cantidad);
                break;
            case 2:
                eliminar(frente, final, cantidad);
                break;
            case 3:
                mostrar(frente);
                break;
            case 4:
                cout << "Saliendo...\n";
                break;
            default:
                cout << "Opcion incorrecta.\n";
        }
    } while (opcion != 4);

    while (frente != nullptr) {
        Nodo* auxiliar = frente;
        frente = frente->siguiente;
        delete auxiliar;
    }

    return 0;
}
```

Pruebas de funcionamiento

Compilar y ejecutar el archivo en el IDE. Probar las opciones disponibles, verificar los resultados y registrar cualquier mensaje de error. Adjuntar una captura auténtica de la ejecución cuando sea requerida.

Análisis de resultados

Registrar aquí los datos de entrada, la salida obtenida y si el comportamiento observado corresponde al funcionamiento esperado de la práctica.

Práctica 7 bis. Lista dinámica con punteros

Introducción

Esta práctica aborda lista dinámica con punteros en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar lista dinámica con punteros mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

Gestiona una lista enlazada mediante punteros.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Archivo fuente original proporcionado: ListaDinamica.cpp. Se conserva el código del estudiante sin sustituirlo por una versión generada.

Código fuente

```
#include <iostream>
using namespace std;

struct Nodo {
    float dato;
    Nodo* siguiente;
};

void insertar(Nodo*& lista, int& cantidad) {
    if (cantidad >= 10) {
        cout << "La lista esta llena.\n";
        return;
    }
```

```
    float valor;
    cout << "Ingresa un numero: ";
    cin >> valor;

    Nodo* nuevo = new Nodo;
    nuevo->dato = valor;
    nuevo->siguiente = nullptr;

    if (lista == nullptr) {
        lista = nuevo;
    } else {
        Nodo* actual = lista;

        while (actual->siguiente != nullptr) {
            actual = actual->siguiente;
        }

        actual->siguiente = nuevo;
    }

    cantidad++;
    cout << "Elemento insertado.\n";
}

void eliminar(Nodo*& lista, int& cantidad) {
    if (lista == nullptr) {
        cout << "La lista esta vacia.\n";
        return;
    }

    float valor;
```

```
    cout << "Valor a eliminar: ";
    cin >> valor;

    Nodo* actual = lista;
    Nodo* anterior = nullptr;

    while (actual != nullptr && actual->dato != valor) {
        anterior = actual;
        actual = actual->siguiente;
    }

    if (actual == nullptr) {
        cout << "Elemento no encontrado.\n";
        return;
    }

    if (anterior == nullptr) {
        lista = actual->siguiente;
    } else {
        anterior->siguiente = actual->siguiente;
    }

    delete actual;
    cantidad--;
    cout << "Elemento eliminado.\n";
}

void mostrar(Nodo* lista) {
    if (lista == nullptr) {
        cout << "La lista esta vacia.\n";
        return;
    }
```

```
    cout << "Elementos: ";
    while (lista != nullptr) {
        cout << lista->dato << " ";
        lista = lista->siguiente;
    }
    cout << endl;
}

int main() {
    Nodo* lista = nullptr;
    int cantidad = 0;
    int opcion;

    do {
        cout << "\n--- LISTA DINAMICA ---\n";
        cout << "1. Insertar\n";
        cout << "2. Eliminar\n";
        cout << "3. Mostrar\n";
        cout << "4. Salir\n";
        cout << "Opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                insertar(lista, cantidad);
                break;
            case 2:
                eliminar(lista, cantidad);
                break;
            case 3:
                mostrar(lista);
```

```
            break;
        case 4:
            cout << "Saliendo...\n";
            break;
        default:
            cout << "Opcion incorrecta.\n";
      }
   } while (opcion != 4);

   while (lista != nullptr) {
      Nodo* auxiliar = lista;
      lista = lista->siguiente;
      delete auxiliar;
   }

   return 0;
}
```

Pruebas de funcionamiento

Compilar y ejecutar el archivo en el IDE. Probar las opciones disponibles, verificar los resultados y registrar cualquier mensaje de error. Adjuntar una captura auténtica de la ejecución cuando sea requerida.

Análisis de resultados

Registrar aquí los datos de entrada, la salida obtenida y si el comportamiento observado corresponde al funcionamiento esperado de la práctica.

Práctica 8. Pila con biblioteca del IDE

Introducción

Esta práctica aborda pila con biblioteca del ide en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar pila con biblioteca del ide mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

Utiliza el contenedor stack de la biblioteca estándar.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Archivo fuente original proporcionado: PilaDinamica.cpp. Se conserva el código del estudiante sin sustituirlo por una versión generada.

Código fuente

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<float> pila;
    int opcion;
    float valor;

    do {
        cout << "\n--- PILA CON STACK ---\n";
        cout << "1. Insertar\n";
        cout << "2. Eliminar\n";
```

```
cout << "3. Mostrar\n";
cout << "4. Salir\n";
cout << "Opcion: ";
cin >> opcion;

switch (opcion) {
    case 1:
        if (pila.size() >= 10) {
            cout << "La pila esta llena.\n";
        } else {
            cout << "Ingresa un numero: ";
            cin >> valor;
            pila.push(valor);
            cout << "Elemento insertado.\n";
        }
        break;

    case 2:
        if (pila.empty()) {
            cout << "La pila esta vacia.\n";
        } else {
            cout << "Eliminado: " << pila.top() << endl;
            pila.pop();
        }
        break;

    case 3: {
        if (pila.empty()) {
            cout << "La pila esta vacia.\n";
        } else {
            stack<float> auxiliar = pila;
```

```
                cout << "Pila (cima a base): ";
                while (!auxiliar.empty()) {
                    cout << auxiliar.top() << " ";
                    auxiliar.pop();
                }
                cout << endl;
            }
            break;
        }

        case 4:
            cout << "Saliendo...\n";
            break;

        default:
            cout << "Opcion incorrecta.\n";
    }

} while (opcion != 4);

return 0;
}
```

Pruebas de funcionamiento

Compilar y ejecutar el archivo en el IDE. Probar las opciones disponibles, verificar los resultados y registrar cualquier mensaje de error. Adjuntar una captura auténtica de la ejecución cuando sea requerida.

Análisis de resultados

Registrar aquí los datos de entrada, la salida obtenida y si el comportamiento observado corresponde al funcionamiento esperado de la práctica.

Práctica 9. Cola con biblioteca del IDE

Introducción

Esta práctica aborda cola con biblioteca del ide en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar cola con biblioteca del ide mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

Utiliza el contenedor queue de la biblioteca estándar.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Archivo fuente original proporcionado: ColaIDE.cpp. Se conserva el código del estudiante sin sustituirlo por una versión generada.

Código fuente

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<float> cola;
    int opcion;
    float valor;

    do {
        cout << "\n--- COLA CON QUEUE ---\n";
        cout << "1. Insertar\n";
        cout << "2. Eliminar\n";
```

```
cout << "3. Mostrar\n";
cout << "4. Salir\n";
cout << "Opcion: ";
cin >> opcion;

switch (opcion) {
    case 1:
        if (cola.size() >= 10) {
            cout << "La cola esta llena.\n";
        } else {
            cout << "Ingresa un numero: ";
            cin >> valor;
            cola.push(valor);
            cout << "Elemento insertado.\n";
        }
        break;

    case 2:
        if (cola.empty()) {
            cout << "La cola esta vacia.\n";
        } else {
            cout << "Eliminado: " << cola.front() << endl;
            cola.pop();
        }
        break;

    case 3: {
        if (cola.empty()) {
            cout << "La cola esta vacia.\n";
        } else {
            queue<float> auxiliar = cola;
```

```
                cout << "Elementos de la cola: ";
                while (!auxiliar.empty()) {
                    cout << auxiliar.front() << " ";
                    auxiliar.pop();
                }
                cout << endl;
            }
            break;
        }

        case 4:
            cout << "Saliendo...\n";
            break;

        default:
            cout << "Opcion incorrecta.\n";
    }

} while (opcion != 4);

return 0;
}
```

Pruebas de funcionamiento

Compilar y ejecutar el archivo en el IDE. Probar las opciones disponibles, verificar los resultados y registrar cualquier mensaje de error. Adjuntar una captura auténtica de la ejecución cuando sea requerida.

Análisis de resultados

Registrar aquí los datos de entrada, la salida obtenida y si el comportamiento observado corresponde al funcionamiento esperado de la práctica.

Práctica 10. Lista con biblioteca del IDE

Introducción

Esta práctica aborda lista con biblioteca del ide en C++. Se estudia el funcionamiento de la estructura o algoritmo y la manera en que se implementa en el programa desarrollado.

Objetivo

Analizar e implementar lista con biblioteca del ide mediante un programa en C++, identificando sus operaciones y el manejo de datos.

Fundamento teórico

Utiliza el contenedor list de la biblioteca estándar.

Materiales y herramientas

Computadora, editor o IDE compatible con C++ y compilador de C++.

Desarrollo

Archivo fuente original proporcionado: ListaIDE.cpp. Se conserva el código del estudiante sin sustituirlo por una versión generada.

Código fuente

```
#include <iostream>
#include <list>
#include <algorithm>
using namespace std;

int main() {
    list<float> lista;
    int opcion;
    float valor;

    do {
        cout << "\n--- LISTA CON LIST ---\n";
        cout << "1. Insertar\n";
```

```
cout << "2. Eliminar\n";
cout << "3. Mostrar\n";
cout << "4. Buscar\n";
cout << "5. Salir\n";
cout << "Opcion: ";
cin >> opcion;

switch (opcion) {
    case 1:
        if (lista.size() >= 10) {
            cout << "La lista esta llena.\n";
        } else {
            cout << "Ingresa un numero: ";
            cin >> valor;
            lista.push_back(valor);
            cout << "Elemento insertado.\n";
        }
        break;

    case 2: {
        if (lista.empty()) {
            cout << "La lista esta vacia.\n";
            break;
        }

        cout << "Valor a eliminar: ";
        cin >> valor;

        auto it = find(lista.begin(), lista.end(), valor);

        if (it != lista.end()) {
            lista.erase(it);
```

```
            cout << "Elemento eliminado.\n";
        } else {
            cout << "Elemento no encontrado.\n";
        }
        break;
    }

    case 3:
        if (lista.empty()) {
            cout << "La lista esta vacia.\n";
        } else {
            cout << "Elementos: ";
            for (float numero : lista) {
                cout << numero << " ";
            }
            cout << endl;
        }
        break;

    case 4:
        cout << "Valor a buscar: ";
        cin >> valor;

        if (find(lista.begin(), lista.end(), valor)
            != lista.end()) {
            cout << "Elemento encontrado.\n";
        } else {
            cout << "Elemento no encontrado.\n";
        }
        break;

    case 5:
```

```
            cout << "Saliendo...\n";
            break;

        default:
            cout << "Opcion incorrecta.\n";
    }

} while (opcion != 5);

return 0;
}
```

Pruebas de funcionamiento

Compilar y ejecutar el archivo en el IDE. Probar las opciones disponibles, verificar los resultados y registrar cualquier mensaje de error. Adjuntar una captura auténtica de la ejecución cuando sea requerida.

Análisis de resultados

Registrar aquí los datos de entrada, la salida obtenida y si el comportamiento observado corresponde al funcionamiento esperado de la práctica.

Archivos adicionales proporcionados

ColasPunteros.cpp

Archivo incluido en el ZIP que no se asignó a una práctica de manera inequívoca. Se conserva íntegro para su revisión.

```
//Programa ColasPunteros sin decimales
#include <iostream>
#include <iomanip>
#include <limits>
#include <string>
#include <sstream>
using namespace std;



struct Nodo {
    double dato;
    Nodo* siguiente;
};

void insertar(Nodo*& frente, Nodo*& final, double valor) {
    Nodo* nuevo = new Nodo;
    nuevo->dato = valor;
    nuevo->siguiente = nullptr;

    if (frente == nullptr) {
        frente = nuevo;
        final = nuevo;
    } else {
        final->siguiente = nuevo;
        final = nuevo;
    }
```

```
    cout << "Elemento insertado correctamente.\n";
}

void eliminar(Nodo*& frente, Nodo*& final) {
    if (frente == nullptr) {
        cout << "La cola esta vacia.\n";
        return;
    }

    Nodo* auxiliar = frente;

    cout << "Elemento eliminado: "
         << frente->dato << endl;

    frente = frente->siguiente;

    if (frente == nullptr) {
        final = nullptr;
    }

    delete auxiliar;
}

void mostrar(Nodo* frente) {
    if (frente == nullptr) {
        cout << "La cola esta vacia.\n";
        return;
    }

    cout << "\nElementos de la cola:\n";

    while (frente != nullptr) {
```

```
        cout << frente->dato << " ";
        frente = frente->siguiente;
    }

    cout << endl;
}

double leerNumero() {
    string entrada;
    double numero;
    char extra;

    while (true) {
        cout << "Ingrese un numero: ";
        cin >> entrada;

            for (char& c : entrada) {
            if (c == ',') {
                c = '.';
            }
        }

        stringstream convertir(entrada);

        if ((convertir >> numero) &&
            !(convertir >> extra)) {
            return numero;
        }

        cout << "Numero no valido. Intente otra vez.\n";
    }
}
```

```
int main() {
    Nodo* frente = nullptr;
    Nodo* final = nullptr;

    int opcion;
    double valor;

    cout << setprecision(10);

    do {
        cout << "\n--- COLA DINAMICA ---\n";
        cout << "1. Insertar\n";
        cout << "2. Eliminar\n";
        cout << "3. Mostrar\n";
        cout << "4. Salir\n";
        cout << "Seleccione una opcion: ";

        if (!(cin >> opcion)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Opcion no valida.\n";
            continue;
        }

        switch (opcion) {
            case 1:
                valor = leerNumero();
                insertar(frente, final, valor);
                break;

            case 2:
```

```
            eliminar(frente, final);
            break;

        case 3:
            mostrar(frente);
            break;

        case 4:
            cout << "Saliendo...\n";
            break;

        default:
            cout << "Opcion no valida.\n";
        }

    } while (opcion != 4);

    while (frente != nullptr) {
    Nodo* auxiliar = frente;
    frente = frente->siguiente;
    delete auxiliar;
    }

    final = nullptr;

    return 0;
}
```

ListaPunteros.cpp

Archivo incluido en el ZIP que no se asignó a una práctica de manera inequívoca. Se conserva íntegro para su revisión.

```
//Programa Lista
#include <iostream>
using namespace std;

const int LIMITE = 10;

struct Nodo {
    int dato;
    Nodo* siguiente;
};
void insertar(Nodo*& lista, int valor) {
    Nodo* nuevo = new Nodo;
    nuevo->dato = valor;
    nuevo->siguiente = nullptr;

    if (lista == nullptr) {
        lista = nuevo;
    } else {
        Nodo* auxiliar = lista;

        while (auxiliar->siguiente != nullptr) {
            auxiliar = auxiliar->siguiente;
        }

        auxiliar->siguiente = nuevo;
    }

    cout << "Elemento insertado correctamente.\n";
}
```

```
void eliminar(Nodo*& lista, int valor) {
    if (lista == nullptr) {
        cout << "La lista esta vacia.\n";
        return;
    }

    Nodo* actual = lista;
    Nodo* anterior = nullptr;

    while (actual != nullptr && actual->dato != valor) {
        anterior = actual;
        actual = actual->siguiente;
    }

    if (actual == nullptr) {
        cout << "Elemento no encontrado.\n";
        return;
    }

    if (anterior == nullptr) {
        lista = actual->siguiente;
    } else {
        anterior->siguiente = actual->siguiente;
    }

    delete actual;
    cout << "Elemento eliminado correctamente.\n";
}

void mostrar(Nodo* lista) {
    if (lista == nullptr) {
```

```
        cout << "La lista esta vacia.\n";
        return;
    }

    cout << "Elementos de la lista:\n";

    while (lista != nullptr) {
        cout << lista->dato << " -> ";
        lista = lista->siguiente;
    }

    cout << "NULL\n";
}

int main() {
    Nodo* lista = nullptr;
    int opcion, valor;

    do {
        cout << "\n--- LISTA DINAMICA ---\n";
        cout << "1. Insertar\n";
        cout << "2. Eliminar\n";
        cout << "3. Mostrar\n";
        cout << "4. Salir\n";
        cout << "Seleccione una opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                cout << "Ingrese un numero: ";
                cin >> valor;
                insertar(lista, valor);
```

```
            break;

        case 2:
            cout << "Ingrese el numero a eliminar: ";
            cin >> valor;
            eliminar(lista, valor);
            break;

        case 3:
            mostrar(lista);
            break;

        case 4:
            cout << "Saliendo...\n";
            break;

        default:
            cout << "Opcion no valida.\n";
    }

} while (opcion != 4);

while (lista != nullptr) {
    Nodo* auxiliar = lista;
    lista = lista->siguiente;
    delete auxiliar;
}

return 0;
}
```